

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

C++ STL 快速入门
六大常用容器与工具

A Quick Start to the C++ Standard Template Library

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. STL 概述
2. vector
3. stack 与 queue
4. priority_queue
5. set
6. map

目录

7 总结

目录

1 STL 概述
什么是 STL
头文件与复杂度记号

2 vector

3 stack 与 queue

4 priority_queue

5 set

目录

6 map

7 总结

什么是 STL

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

STL = Standard Template Library

由四大组件构成：

- **容器**：存数据（vector、map）
- **迭代器**：像指针，用来遍历
- **算法**：sort、reverse、find
- **函数对象**：greater、less

为什么必须学 STL？

- 现成的数据结构，不必手写
- 经过工业级优化，性能好
- 代码更短，更少 bug
- 竞赛与工程都通用

一句话

把“造轮子”的时间省下来，用来想算法。

常用头文件与复杂度记号

头文件	提供的内容
`<vector>`	动态数组
`<stack>`	栈
`<queue>`	队列、优先队列
`<set>`	有序集合
`<map>`	有序映射
`<algorithm>`	通用算法

复杂度记号

- $O(1)$：一步到位，最快
- $O(\log n)$：砍半查找，很快
- $O(n)$：看一遍所有元素
- $O(n \log n)$：排序级别
- $O(n^2)$：双重循环，慎用

什么是“均摊 O(1)”？

大多数操作是 $O(1)$，偶尔一次是 $O(n)$，分摊到很多次上，平均还是 $O(1)$。

代码：最简 STL 程序框架


```cpp
// 万能头文件：竞赛中一行包含所有 STL
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. 声明容器
    vector<int> v;                  // 创建空的可变长数组

    // 2. 使用容器
    v.push_back(1);                 // 在末尾添加 1
    v.push_back(2);                 // 在末尾添加 2

    // 3. 配合算法
    sort(v.begin(), v.end());           // 把 v 从小到大排序

    for (int x : v) cout << x << " ";  // 依次输出 v 中每个元素
    cout << endl;
    return 0;
```

代码：最简 STL 程序框架


```
19 }
```

目录

1 STL 概述

2 vector
- 特点与操作
- 代码示例

3 stack 与 queue

4 priority_queue

5 set

目录

6 map

7 总结

vector：特点与操作

特点

- 长度可自动增长
- 支持随机访问 `v[i]`，$O(1)$
- 尾部增删 $O(1)$ 均摊
- 中间增删 $O(n)$
- 最常用，默认首选

声明方式

- `vector<int> v;` 空数组
- `vector<int> v(5, 3);` 5 个 3
- `vector<int> v = {1,2,3};`

常用操作说明与复杂度

操作	说明	复杂度
`push_back(x)`	在末尾添加 x	$O(1)$ 均摊
`pop_back()`	删除末尾元素	$O(1)$
`v[i]`	访问下标 i 的元素	$O(1)$
`front()/back()`	首元素 / 尾元素	$O(1)$
`size()/empty()`	元素个数 / 判空	$O(1)$
`clear()`	清空所有元素	$O(n)$
`resize(n)`	把长度改为 n	$O(n)$
`reserve(n)`	预分配容量 n	$O(n)$

代码：vector 基础操作


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. 四种初始化
    vector<int> a;                  // 空数组
    vector<int> b(5);               // 5 个 0
    vector<int> c(5, 3);            // 5 个 3
    vector<int> d = {1, 2, 3, 4};   // 列表初始化

    // 2. 尾部增删
    a.push_back(10);                // 在末尾添加 10
    a.push_back(20);                // 在末尾添加 20
    a.pop_back();                   // 删除末尾元素 (20)

    // 3. 元素访问
    cout << a[0] << endl;           // 用下标访问第 0 个元素
    cout << a.at(0) << endl;        // 带越界检查的访问
```

代码：vector 基础操作


```
19      cout << a.front() << " " << a.back() << endl;
20
21      // 4. 容量相关
22      cout << a.size() << " " << a.empty() << endl;
23      a.resize(4, 7);                 // 把长度改为 4，多出的填 7
24      a.clear();                      // 清空所有元素
25
26      // 5. reserve：预分配容量，避免反复扩容
27      vector<int> e;
28      e.reserve(1000);                // 预留 1000 个元素的空间
29      for (int i = 0; i < 1000; i++) e.push_back(i);
30      return 0;
31  }
```

代码：vector 五种遍历方式


```cpp
vector<int> v = {3, 1, 4, 1, 5};

// 方式一：下标，逐个访问
for (int i = 0; i < (int)v.size(); i++) cout << v[i] << " ";

// 方式二：显式迭代器，it 像指针
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
    cout << *it << " ";

// 方式三：auto 迭代器（推荐）
for (auto it = v.begin(); it != v.end(); ++it) cout << *it << " ";

// 方式四：范围 for，只读
for (int x : v) cout << x << " ";

// 方式五：范围 for，加引用可修改
for (int &x : v) x *= 2;              // 每个元素乘以 2

```

代码：vector 五种遍历方式


```
19 // 排序 + 去重 (经典组合)
20 sort(v.begin(), v.end());                          // 从小到大排序
21 v.erase(unique(v.begin(), v.end()), v.end());     // 去掉重复元素
```

代码：二维 vector 与邻接表


```cpp
// 1. n 行 m 列的二维数组，初值全 0
int n = 3, m = 4;
vector<vector<int>> g(n, vector<int>(m, 0));

g[1][2] = 5;                                    // 修改第 1 行第 2 列

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) cout << g[i][j] << " ";
    cout << endl;
}

// 2. 不定长二维数组：图的邻接表
vector<vector<int>> adj(n);                     // n 个空列表
adj[0].push_back(1);                            // 0 号点连到 1 号点
adj[0].push_back(2);                            // 0 号点连到 2 号点
adj[1].push_back(0);                            // 1 号点连到 0 号点

for (int i = 0; i < n; i++) {
```

代码：二维 vector 与邻接表


```
19      cout << i << " -> ";
20      for (int x : adj[i]) cout << x << " ";
21      cout << endl;
22  }
```

目录

1 STL 概述

2 vector

3 stack 与 queue
- stack 栈
- queue 队列

4 priority_queue

5 set

目录

6 map

7 总结

stack 与 queue：特点与操作

stack（栈）：后进先出 LIFO

- `push(x)`：压入栈顶，$O(1)$
- `pop()`：弹出栈顶，$O(1)$
- `top()`：查看栈顶，$O(1)$
- `size()/empty()`，$O(1)$
- 无迭代器，不能随机访问

典型用途：括号匹配、表达式求值、DFS、单调栈。

queue（队列）：先进先出 FIFO

- `push(x)`：加入队尾，$O(1)$
- `pop()`：弹出队头，$O(1)$
- `front()`：查看队头，$O(1)$
- `back()`：查看队尾，$O(1)$
- 无迭代器，不能随机访问

典型用途：任务调度、模拟排队。

共同限制：不能用范围 for，想遍历只能反复 front()/top() 再 pop()，整体 $O(n)$。

代码：stack 基础与应用


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    stack<int> st;
    st.push(1);                  // 把 1 压入栈顶
    st.push(2);                  // 把 2 压入栈顶
    st.push(3);                  // 把 3 压入栈顶

    cout << st.top() << endl;    // 查看栈顶，输出 3
    cout << st.size() << endl;   // 元素个数，输出 3

    st.pop();                    // 弹出栈顶 (3)
    cout << st.top() << endl;    // 现在栈顶是 2

    while (!st.empty()) {        // 只要栈不空
        cout << st.top() << " "; // 输出栈顶
        st.pop();                // 弹出栈顶
```

代码：stack 基础与应用


```
    }                                   // 输出 2 1
    cout << endl;
    return 0;
}

// 应用：判断括号是否匹配，O(n)
bool check(const string &s) {
    stack<char> st;
    string L = "([{", R = ")]}";
    for (char c : s) {                              // 遍历每个字符
        size_t li = L.find(c);                      // 是左括号吗？
        if (li != string::npos) { st.push(c); continue; }
        size_t ri = R.find(c);                      // 是右括号吗？
        if (ri == string::npos) continue;           // 都不是，跳过
        if (st.empty()) return false;               // 右括号多了
        if (st.top() != L[ri]) return false;        // 类型不匹配
        st.pop();                                   // 匹配成功，弹出
    }
```

代码：stack 基础与应用


```
37      return st.empty();                          // 左括号必须全部匹配完
38  }
```

代码：queue 基础操作

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    queue<int> q;
    q.push(1);                  // 把 1 加入队尾
    q.push(2);                  // 把 2 加入队尾
    q.push(3);                  // 把 3 加入队尾

    cout << q.front() << endl;  // 查看队头，输出 1
    cout << q.back() << endl;   // 查看队尾，输出 3
    cout << q.size() << endl;   // 元素个数，输出 3

    q.pop();                    // 弹出队头 (1)
    cout << q.front() << endl;  // 现在队头是 2

    while (!q.empty()) {        // 只要队列不空
        cout << q.front() << " ";  // 输出队头
```

代码：queue 基础操作


```
19          q.pop();                    // 弹出队头
20      }                               // 输出 2 3
21      cout << endl;
22      return 0;
23  }
```

目录

1 STL 概述

2 vector

3 stack 与 queue

4 priority_queue
- 特点与操作
- 三种常用声明
- 比较规则
- 代码示例



目录

5 set

6 map

7 总结

priority_queue：特点与操作

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

特点

- 每次取出的都是“优先级最高”的元素
- 底层是二叉堆
- 插入 / 删除 $O(\log n)$
- 取堆顶 $O(1)$，但只能取堆顶
- 无迭代器，不能遍历

常用操作说明与复杂度

操作	说明	复杂度
`push(x)`	插入元素 x	$O(\log n)$
`pop()`	删除堆顶元素	$O(\log n)$
`top()`	查看堆顶元素	$O(1)$
`size()`	返回元素个数	$O(1)$
`empty()`	判断是否为空	$O(1)$
建堆（n 个元素）	一次性构建堆	$O(n)$
堆排序	不断弹出堆顶	$O(n \log n)$

priority_queue：三种常用声明

1. 大顶堆（默认）

最大值在堆顶。

```
1 priority_queue<int> pq;
2
```

2. 小顶堆

最小值在堆顶，加 greater 和底层容器。

```
1 priority_queue<int, vector<int>,
2                greater<int>> pq;
3
```

3. 自定义类型

在结构体里重载 operator<。

```
1 struct Node {
2     int dist;
3     bool operator<(const Node &o)
4         const {
5         return dist > o.dist;
6     }
7 };
8 priority_queue<Node> pq;
9
```

priority_queue：比较规则（重点）

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

sort：返回 a < b 表示升序

```
sort(v.begin(), v.end(),
     [](int a, int b) {
         return a < b;  // 小的在前
     });

```

priority_queue：返回 a < b 表示 b 优先

```
struct Node {
    bool operator<(const Node &o)
        const {
        return x < o.x;
        // 大的优先（大顶堆）
    }
};

```

一句话记忆：sort 的 < 是“升序”，priority_queue 的 < 是“优先级低”，方向正好相反。

代码：priority_queue 三种写法


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. 默认大顶堆：最大值在堆顶
    priority_queue<int> pq;
    pq.push(3);                    // 插入 3
    pq.push(1);                    // 插入 1
    pq.push(4);                    // 插入 4
    cout << pq.top() << endl;      // 查看堆顶，输出 4
    pq.pop();                      // 删除堆顶 (4)

    // 2. 小顶堆：最小值在堆顶
    priority_queue<int, vector<int>, greater<int>> small;
    small.push(3);                 // 插入 3
    small.push(1);                 // 插入 1
    small.push(4);                 // 插入 4
    cout << small.top() << endl;   // 查看堆顶，输出 1
```

代码：priority_queue 三种写法


```
19
20      // 3. 用负号把大顶堆变成小顶堆（技巧）
21      priority_queue<int> trick;
22      trick.push(-3);
23      trick.push(-1);
24      cout << -trick.top() << endl;   // 输出 1
25
26      // 4. 堆排序效果：不断取堆顶
27      while (!pq.empty()) {
28          cout << pq.top() << " ";
29          pq.pop();
30      }
31      return 0;
32  }
```

代码：自定义比较与 Top-K


```cpp
// 自定义结构体：重载 operator<
struct Node {
    int id, dist;
    // dist 越大优先级越低 -> 小顶堆
    bool operator<(const Node &o) const {
        return dist > o.dist;      // dist 小的排上面
    }
};

int main() {
    priority_queue<Node> pq;
    pq.push({1, 30});
    pq.push({2, 10});
    pq.push({3, 20});
    cout << pq.top().id << endl;   // 堆顶是 dist 最小的，输出 2

    // 应用：求数组中最大的 K 个数（小顶堆维护）
    vector<int> a = {5, 1, 9, 3, 7, 8};
```

代码：自定义比较与 Top-K


```
19      int K = 3;
20      priority_queue<int, vector<int>, greater<int>> heap;
21      for (int x : a) {
22          heap.push(x);                // 插入小顶堆
23          if ((int)heap.size() > K) heap.pop();   // 超过 K 个弹掉最小的
24      }
25      while (!heap.empty()) {
26          cout << heap.top() << " "; // 从小到大输出
27          heap.pop();
28      }
29      return 0;
30  }
```

目录

1 STL 概述

2 vector

3 stack 与 queue

4 priority_queue

5 set

特点与操作
代码示例



目录

6 map

7 总结

set：特点与操作

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

特点

- 元素**自动去重**
- 元素**自动升序**排列
- 底层是红黑树
- 插入 / 查找 / 删除都是 $O(\log n)$

变体

- `multiset`：允许重复
- `unordered_set`：平均 $O(1)$
- `set<int, greater<int>>`：降序

常用操作说明与复杂度

操作	说明	复杂度
`insert(x)`	插入 x（自动去重排序）	$O(\log n)$
`erase(x)`	删除值为 x 的元素	$O(\log n)$
`find(x)`	查找 x，返回迭代器	$O(\log n)$
`count(x)`	返回 x 的个数	$O(\log n)$
`lower_bound(x)`	第一个 $\geq x$	$O(\log n)$
`upper_bound(x)`	第一个 $> x$	$O(\log n)$
遍历	按升序访问	$O(n)$
`size()/empty()`	容量信息	$O(1)$

代码：set 完整用法


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    set<int> s;

    // 1. 插入：自动去重 + 自动排序
    s.insert(3);                    // 插入 3
    s.insert(1);                    // 插入 1
    s.insert(4);                    // 插入 4
    s.insert(1);                    // 重复插入 1，被忽略
    cout << s.size() << endl;       // 元素个数，输出 3

    // 2. 遍历：一定是升序
    for (int x : s) cout << x << " ";   // 输出 1 3 4
    cout << endl;

    // 3. 查找
```

代码：set 完整用法


```
19      auto it = s.find(3);          // 查找 3
20      if (it != s.end()) cout << "found " << *it << endl;
21
22      // 4. count：存在返回 1，不存在返回 0
23      cout << s.count(3) << " " << s.count(9) << endl;   // 输出 1 0
24
25      // 5. 边界查找（有序容器的杀手锏）
26      set<int> t = {10, 20, 30, 40};
27      cout << *t.lower_bound(20) << endl;   // 第一个 >= 20，输出 20
28      cout << *t.upper_bound(20) << endl;   // 第一个 > 20，输出 30
29
30      // 6. 删除
31      s.erase(1);                   // 删除值为 1 的元素
32      s.erase(s.begin());           // 删除第一个元素
33      s.clear();                    // 清空所有元素
34
35      // 7. 降序 set
36      set<int, greater<int>> ds = {1, 3, 2};
```

代码：set 完整用法


```
37      for (int x : ds) cout << x << " ";     // 输出 3 2 1
38      cout << endl;
39      return 0;
40  }
```

目录

1 STL 概述

2 vector

3 stack 与 queue

4 priority_queue

5 set

6 map



目录

特点与操作
最容易踩的坑
代码示例

7 总结

map：特点与操作

特点

- 存 `key -> value` 的对应关系
- key **自动去重并按升序**排列
- **底层是红黑树**
- **插入 / 查找 / 删除都是** $O(\log n)$
- **元素类型是键值对**

变体

- `multimap`：key **可重复**
- `unordered_map`：**平均** $O(1)$

常用操作说明与复杂度

操作	说明	复杂度
`mp[k] = v`	把 key k 的值设为 v	$O(\log n)$
`mp.at(k)`	取 key k 的值（带检查）	$O(\log n)$
`insert({k,v})`	插入键值对	$O(\log n)$
`find(k)`	查找 key k	$O(\log n)$
`count(k)`	返回 key k 的个数	$O(\log n)$
`erase(k)`	删除 key k	$O(\log n)$
遍历	按 key 升序访问	$O(n)$
`size()/empty()`	容量信息	$O(1)$

map 最容易踩的坑

错误示范：用 [] 查询

```cpp
map<int,int> mp;
cout << mp.size();   // 0
int x = mp[999];
// 静默插入 999 -> 0
cout << mp.size();   // 1

```

正确做法：find / count

```cpp
if (mp.find(999) != mp.end()) {
    // 存在才操作
}
if (mp.count(999)) {
    // 存在
}

```

mp[key] 在 key 不存在时会**自动插入**默认值，只查询请用 find 或 count。

代码：map 完整用法


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    map<int, int> mp;

    // 1. 插入：三种方式
    mp[1] = 5;                          // 把 key 1 映射到 5
    mp.insert({2, 3});                  // 插入键值对 (2, 3)
    mp.insert(make_pair(3, 7));         // 同上，用 make_pair

    // 2. 访问
    cout << mp[1] << endl;              // 取 key 1 的值，输出 5
    cout << mp.at(2) << endl;           // 取 key 2 的值，输出 3

    // 3. 安全查找：不要用 []
    if (mp.find(3) != mp.end())         // 查找 key 3
        cout << mp[3] << endl;
```

代码：map 完整用法


```
19      if (mp.count(4) == 0)
20          cout << "not found" << endl;
21
22      // 4. 遍历：按 key 升序
23      for (auto &kv : mp)                 // kv 是键值对
24          cout << kv.first << " -> " << kv.second << endl;
25
26      // 5. 删除
27      mp.erase(1);                        // 删除 key 为 1 的项
28      auto it = mp.find(2);               // 找到 key 2
29      if (it != mp.end()) mp.erase(it);   // 按迭代器删除
30
31      // 6. 降序 map
32      map<int, int, greater<int>> dm;
33      dm[1] = 10; dm[3] = 30; dm[2] = 20;
34      for (auto &kv : dm) cout << kv.first << " ";   // 输出 3 2 1
35      cout << endl;
36
```

代码：map 完整用法


```
37      // 7. 哈希版：平均 O(1)，无序
38      unordered_map<int, int> um;
39      um[5] = 50;
40      return 0;
41  }
```

代码：map 典型应用——数字频次统计


```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 统计每个数字出现的次数
    vector<int> nums = {1, 2, 3, 1, 2, 1, 4, 5, 4, 1};
    map<int, int> freq;

    for (int x : nums) {
        freq[x]++;              // 计数加 1（不存在则自动插入 0 再自增）
    }

    for (auto &kv : freq)       // 按 key 升序遍历
        cout << kv.first << " : " << kv.second << endl;

    // 输出（按 key 升序）：
    // 1 : 4
    // 2 : 2
```

代码：map 典型应用——数字频次统计


```
19      // 3 : 1
20      // 4 : 2
21      // 5 : 1
22      return 0;
23  }
```

目录

1 STL 概述

2 vector

3 stack 与 queue

4 priority_queue

5 set

6 map

目录

7 总结
容器选择速查
复杂度与常见陷阱

容器选择速查表（一）：线性容器

需求	推荐容器	关键操作	复杂度
动态数组、随机访问	`vector`	尾插、下标访问	尾插 $O(1)$ 均摊，访问 $O(1)$
后进先出	`stack`	压栈、弹栈、看栈顶	均 $O(1)$
先进先出	`queue`	入队、出队、看队头	均 $O(1)$

容器选择速查表（二）：树与哈希容器

需求	推荐容器	关键操作	复杂度
每次取最值	`priority_queue`	插入、弹出堆顶、看堆顶	插入/弹出 $O(\log n)$，看 $O(1)$
有序去重集合	`set`	插入、查找、删除	均 $O(\log n)$
键值映射有序	`map`	插入、查找、删除	均 $O(\log n)$
只要去重	`unordered_set`	插入、查找、删除	平均 $O(1)$
只要映射	`unordered_map`	插入、查找、删除	平均 $O(1)$

复杂度对比与选容器口诀

复杂度对比

容器	插入	查找	删除
vector	尾 $O(1)$	$O(n)$	尾 $O(1)$
set / map	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_*	均摊 $O(1)$	均摊 $O(1)$	均摊 $O(1)$
priority_queue	$O(\log n)$	堆顶 $O(1)$	$O(\log n)$

选容器口诀

- 随机访问用 `vector`
- 自动排序用 `set/map`
- 每次取最值用 `priority_queue`
- 只要去重用 `unordered_set`

常见陷阱（务必记住）

1. `v.size()` 是无符号数，空容器做 `size()-1` 会溢出，比较前先转（`int`）。
2. `mp[key]` 在 `key` 不存在时会**自动插入**默认值，只查询请用 `find` 或 `count`。
3. `priority_queue` 的比较方向与 `sort` 相反。
4. `stack` / `queue` / `priority_queue` 没有迭代器。
5. 容器扩容后旧迭代器会失效（如 `vector::push_back`）。
6. `set` 的元素是只读的，要改只能先删再插。
7. `unordered_*` 最坏情况可能退化为 $O(n)$。

附录：常见陷阱与修复

```cpp
// 陷阱 1：size() 是无符号数，空容器减法溢出
vector<int> v;
// for (int i = 0; i <= v.size() - 1; i++)   // 危险！
for (int i = 0; i < (int)v.size(); i++) { }   // 正确，先转成 int

// 陷阱 2：map 用 [] 查询会插入新元素
map<int, int> mp;
cout << mp.size() << endl;   // 0
int x = mp[999];             // 静默插入 999 -> 0，size 变 1
cout << mp.size() << endl;   // 1

// 正确做法：
if (mp.find(999) != mp.end()) { /* 存在 */ }
if (mp.count(999)) { /* 存在 */ }

// 陷阱 3：priority_queue 与 sort 比较规则相反
// sort：返回 a < b 表示升序
// priority_queue：返回 a < b 表示 b 优先级更高（大顶堆）
```

附录：常见陷阱与修复


```cpp
struct Node {
    int v;
    bool operator<(const Node &o) const {
        return v > o.v;       // v 小的优先级高 -> 小顶堆
    }
};

// 陷阱 4: 迭代器失效
vector<int> a = {1, 2, 3};
auto it = a.begin();
a.push_back(4);               // 可能触发扩容
// cout << *it;               // 危险! it 可能已失效
```

附录：学习路线建议

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

第一步：会用

- 背下每个容器的 5 个核心函数
- 手写一遍本讲的示例代码
- 遇到问题先查 cppreference

第二步：用对

- 根据需求选容器（见速查表）
- 关注复杂度，避免超时
- 记住各容器的限制

第三步：用巧

- 与 `<algorithm>` 配合：`sort`、`unique`、`lower_bound`
- 用 lambda 写自定义比较

一句话总结

先想清楚“要什么操作最快”，再查操作表选容器，最后才写代码。